

STOCKIQ · DOCUMENTAÇÃO DO FRONTEND

Frontend Aplicação Web

Documentação da interface do StockIQ: arquitetura, design system, mapa de telas, componentes e integração com a API.

Next.js 16 · App Router

React 19

TypeScript

Tailwind CSS 4

Recharts

Gabriel da Silveira Pessoni · Lívia Portela Ferreira

FATEC Franca — Desenvolvimento de Software Multiplataforma ·
6º semestre

Projeto Integrador VI

Computação em Nuvem II · Mineração
de Dados

Sumário

01	Visão geral	Propósito e números do front-end
02	Stack tecnológica	Frameworks e bibliotecas
03	Arquitetura da aplicação	App Router, grupos de rota e camadas
04	Estrutura de pastas	Organização do código
05	Design system	Tokens, tema claro/escuro e tipografia
06	Layout e navegação	AppShell, sidebar e topbar
07	Componentes reutilizáveis	Biblioteca de UI
08	Mapa de telas	Todas as rotas e seus objetivos
09	Telas em destaque	Dashboard, produtos, IA analítica, scanner
10	Autenticação e sessão	Fluxo de login e proteção de rotas
11	Integração com a API	Cliente HTTP, tipos e mapeamento
12	Execução e convenções	Scripts, variáveis e padrões de código

01 Visão geral

O front-end do StockIQ é uma aplicação web em Next.js/React que dá forma à operação do armazém: cadastros, movimentações, lotes, inventário, alertas, relatórios e inteligência analítica, em uma interface limpa, responsiva e com tema claro e escuro.

25

ROTAS

14

COMPONENTES DE UI

6

GRUPOS DE MENU

2

TEMAS

A interface consome a API REST do backend por HTTP/JSON, com autenticação por token JWT. Todo o sistema compartilha um único design system (tokens CSS, componentes e layout), o que garante consistência visual entre as telas e facilita a evolução do produto.

**Design system próprio**

Tokens de cor, raio, sombra e espaçamento centralizados em CSS variables.

**Tema claro e escuro**

Alternância persistida no navegador, respeitando a preferência do sistema.

**Responsivo**

Sidebar recolhível no desktop e gaveta no mobile; layouts em grid adaptativo.

**Fluxo de operação completo**

Do cadastro ao relatório, passando por entradas, saídas, lotes e inventário.

**Dados em tempo real**

Cadastros principais integrados à API com estados de carregamento e erro.

**Inteligência analítica**

Telas de IA para previsão de demanda, matriz ABC/XYZ e sugestões de compra.

02 Stack tecnológica

Camada	Tecnologia	Finalidade
Framework	Next.js 16.2 (App Router)	Roteamento por pastas, layouts aninhados e renderização híbrida
UI	React 19	Componentização e hooks
Linguagem	TypeScript 5	Tipagem de domínio compartilhada entre telas, mocks e API
Estilização	Tailwind CSS 4 + CSS variables	Utilitários de estilo sobre tokens do design system
Ícones	lucide-react	Conjunto único de ícones vetoriais (traço 1.75)
Gráficos	Recharts 3	Barras, linhas e áreas dos dashboards e da IA analítica
Fontes	Geist Sans · Geist Mono	Tipografia carregada via next/font
Qualidade	ESLint 9 (eslint-config-next)	Padronização e checagem estática
Utilitários	clsx	Composição condicional de classes (helper <code>cn</code>)

03 Arquitetura da aplicação

A aplicação usa o App Router do Next.js. Rotas públicas e autenticadas ficam em grupos de layout separados, e o estado de sessão é fornecido por um `AuthProvider` no layout raiz.

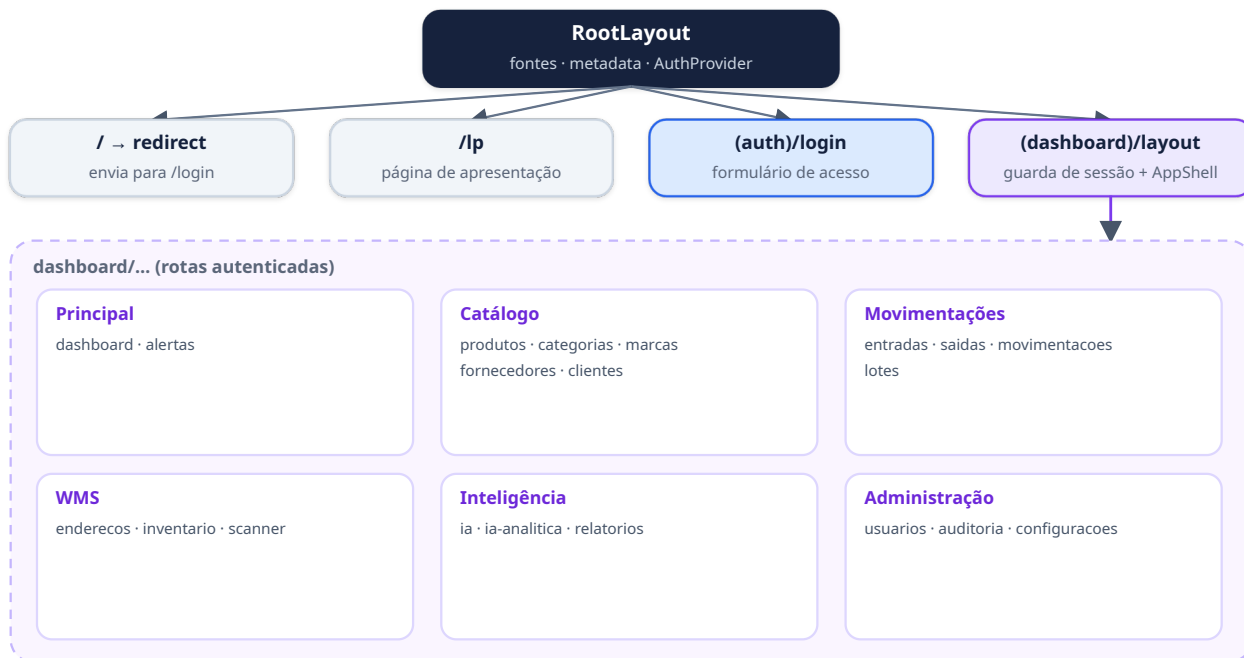


Figura 1 — Árvore de rotas e grupos de layout

Camadas

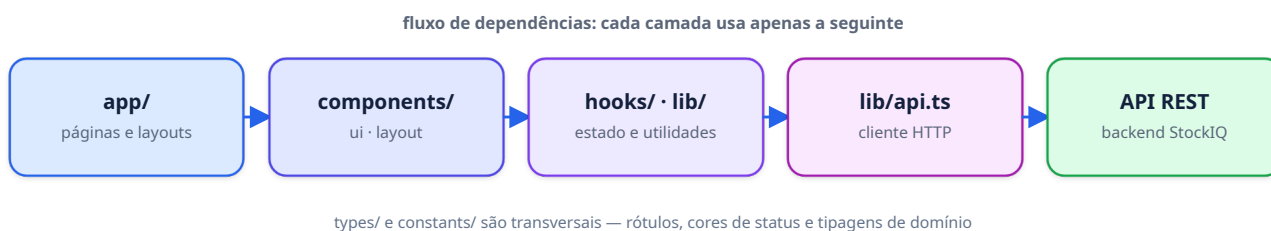


Figura 2 — Camadas do front-end

04 Estrutura de pastas

```

frontend/
├── app/
│   ├── layout.tsx · page.tsx · globals.css    # raiz, redirect e design tokens
│   ├── lp/page.tsx                          # página de apresentação do produto
│   ├── (auth)/login/                         # autenticação
│   └── (dashboard)/
│       ├── layout.tsx                       # guarda de sessão + AppShell
│       └── dashboard/
│           ├── page.tsx                     # dashboard geral
│           ├── produtos/ (lista · novo · [id]) # catálogo
│           ├── categorias/ marcas/ fornecedores/ clientes/
│           ├── entradas/ saidas/ movimentacoes/ lotes/
│           ├── enderecos/ inventario/ scanner/
│           ├── ia/ ia-analitica/ relatorios/
│           └── usuarios/ auditoria/ alertas/ configuracoes/
├── components/
│   ├── layout/ # AppShell · Sidebar · Topbar
│   └── ui/     # 14 componentes reutilizáveis
├── constants/ # navigation.ts · status.ts
├── hooks/     # useDebounce · useSidebar · useTheme
├── lib/       # api.ts · auth-context.tsx · cn.ts · utils.ts
├── mocks/     # dados de demonstração por domínio
└── types/     # product · movement · user · warehouse · common

```

Pasta	Responsabilidade
<code>app/</code>	Rotas, layouts e páginas. Grupos <code>(auth)</code> e <code>(dashboard)</code> separam telas públicas das autenticadas
<code>components/ui</code>	Componentes genéricos, sem regra de negócio
<code>components/layout</code>	Estrutura de página compartilhada: shell, menu lateral e barra superior
<code>constants/</code>	Itens de navegação e dicionários de rótulos/cores de status
<code>hooks/</code>	Hooks utilitários: debounce, estado da sidebar e tema
<code>lib/</code>	Cliente HTTP, contexto de autenticação, helper de classes e formataadores
<code>types/</code>	Tipagens de domínio compartilhadas
<code>mocks/</code>	Conjuntos de dados de demonstração (dashboard, movimentações, produtos, usuários, armazém)

05 Design system

O visual é definido por tokens em `app/globals.css`, expostos como CSS variables e consumidos pelos componentes via Tailwind. O tema escuro redefine os mesmos tokens sob a classe `.dark`, sem duplicar componentes.

● Paleta

Token	Claro	Escuro	Uso
<code>--brand</code>	 #2563eb	 #2563eb	Ações primárias, links, destaque
<code>--success</code>	 #16a34a	 #16a34a	Estoque ok, entradas, confirmações
<code>--warning</code>	 #d97706	 #d97706	Estoque baixo, vencimento próximo
<code>--danger</code>	 #dc2626	 #dc2626	Ruptura, perdas, exclusões
<code>--info</code>	 #0891b2	 #0891b2	Saídas, informações neutras
<code>--bg-base</code>	 #ffffff	 #1c1c1e	Superfície de cartões
<code>--bg-subtle</code>	 #fafafa	 #151517	Fundo da página
<code>--text-primary</code>	 #1d1d1f	 #f5f5f7	Texto principal
<code>--text-secondary</code>	 #6e6e73	 #a1a1a6	Texto de apoio
<code>--border</code>	 #e9e9ec	 #2c2c2e	Bordas e divisores

● Dimensões, raio e elevação

Grupo	Valores
Sidebar	224 px expandida · 64 px recolhida
Topbar	44 px de altura
Conteúdo	largura máxima de 1360 px
Raio	sm 6 · md 8 · lg 12 · xl 16 · 2xl 20 · full
Sombras	sm · md · lg · xl — suaves e quase imperceptíveis
Transição	200 ms · cubic-bezier(0.4, 0, 0.2, 1)
Tipografia	Geist Sans (texto) e Geist Mono (códigos, SKUs)

● Cores semânticas de status

Domínio	Mapeamento
Estoque	ok · Em Estoque low · Estoque Baixo critical · Crítico out · Sem Estoque
Movimentação	Entrada Saída Transferência Perda Ajuste Inventário
Produto	Ativo Inativo Descontinuado
Perfil	Administrador · Supervisor · Operador · Visualizador

TEMA

O hook `useTheme` lê a preferência salva em `localStorage` (ou a do sistema operacional), aplica a classe `dark` no elemento `<html>` e persiste a escolha ao alternar.

06 Layout e navegação

Todas as telas autenticadas compartilham o `AppShell` : uma sidebar agrupada por contexto de uso, uma topbar fina com busca global (atalho `Ctrl/⌘ + K`), alertas, tema e perfil, e uma área de conteúdo centralizada.

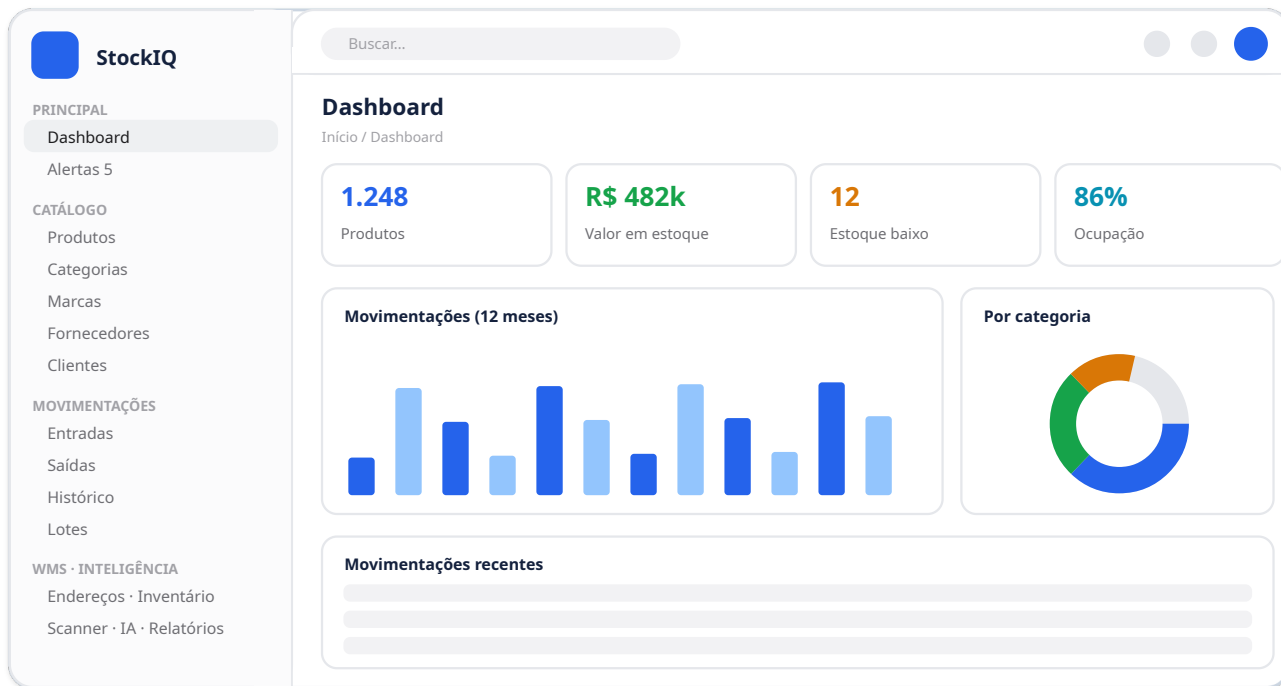


Figura 3 — Estrutura do AppShell (sidebar + topbar + conteúdo)

Menu lateral

Grupo	Itens
Principal	Dashboard · Alertas
Catálogo	Produtos · Categorias · Marcas · Fornecedores · Clientes
Movimentações	Entradas · Saídas · Histórico · Lotes
WMS	Endereços · Inventário · Scanner
Inteligência	IA — Produtos · IA Analítica · Relatórios
Administração	Usuários · Auditoria · Configurações

A sidebar pode ser recolhida (64 px, apenas ícones com *tooltip*), e, no mobile, abre como gaveta sobreposta. O estado é controlado pelo hook `useSidebar` . O item ativo é destacado pelo caminho atual da rota, e a seção do usuário exibe avatar, nome e ação de sair.

07 Componentes reutilizáveis

A biblioteca em `components/ui` reúne 14 componentes sem regra de negócio, todos estilizados pelos tokens do design system e com suporte automático aos temas claro e escuro.

Componente	Descrição	Variações e recursos
<code>Button</code>	Botão de ação	primary · secondary · ghost · danger · outline · tamanhos xs–lg · estado <i>loading</i> · ícones à esquerda/direita
<code>Badge</code>	Etiqueta de status	Variantes semânticas (success, warning, danger, info, brand, default, secondary) · tamanhos · ponto indicador
<code>Card</code>	Contêiner de conteúdo	CardHeader · CardTitle com descrição · preenchimento opcional
<code>StatCard</code>	Indicador numérico	Valor, rótulo, ícone, tendência e variantes de cor
<code>DataTable</code>	Tabela de dados	Colunas configuráveis, ordenação, linhas clicáveis e mensagem de lista vazia
<code>Pagination</code>	Paginação	Navegação por páginas com total de itens
<code>Input</code> · <code>Select</code>	Campos de formulário	Rótulo, ícones, mensagem de erro e ajuda
<code>Modal</code>	Diálogo sobreposto	Título, descrição, rodapé de ações e tamanhos sm a full
<code>Tabs</code>	Abas	Troca de painéis no mesmo contexto
<code>Breadcrumb</code>	Trilha de navegação	Itens com links e item atual
<code>Avatar</code>	Identidade do usuário	Iniciais ou imagem
<code>EmptyState</code>	Estado vazio	Ícone, título, descrição e ação
<code>Skeleton</code>	Carregamento	Placeholders animados

● Hooks e utilitários

Item	Função
<code>useDebounce</code>	Adia a propagação de um valor, útil em buscas para evitar processamento a cada tecla
<code>useSidebar</code>	Controla sidebar recolhida e gaveta mobile
<code>useTheme</code>	Alterna e persiste o tema claro/escuro
<code>useAuth</code>	Acessa usuário, estado de carregamento, <code>login</code> e <code>logout</code>
<code>cn()</code>	Compõe classes Tailwind condicionais (clsx)
<code>formatCurrency</code> <code>formatNumber</code>	Formatação pt-BR de moeda e números

08 Mapa de telas

O sistema reúne 25 rotas. As telas de cadastro principais já operam diretamente sobre a API; as demais oferecem a experiência completa da operação, preparada para receber os mesmos endpoints.

API real dados lidos e gravados na API **Interface completa** fluxo completo de interface com dados de demonstração

Acesso

Rota	Descrição	Dados
/	Redireciona para o login	—
/lp	Página de apresentação do produto: recursos, módulos e chamada para ação	—
/login	Acesso com e-mail e senha; painel de marca com os diferenciais	API real

Visão geral

Rota	Descrição	Dados
/dashboard	Indicadores, tendência mensal, distribuição por categoria, movimentações recentes	Interface completa
/dashboard/alertas	Alertas de estoque e vencimento por severidade	Interface completa

Catálogo

Rota	Descrição	Dados
/dashboard/produtos	Listagem em tabela ou cartões, busca, filtros e exclusão	API real
/dashboard/produtos/novo	Cadastro com categoria, marca, fornecedor, preços e limites	API real
/dashboard/produtos/[id]	Detalhe completo do produto	Interface completa
/dashboard/categorias	CRUD com cor da categoria	API real
/dashboard/marcas	CRUD de marcas	API real
/dashboard/fornecedores	Cadastro de fornecedores	API real
/dashboard/clientes	Cadastro de clientes	API real

Movimentações

Rota	Descrição	Dados
/dashboard/entradas	Registro de entradas de estoque	Interface completa
/dashboard/saidas	Registro de saídas de estoque	Interface completa
/dashboard/movimentacoes	Histórico geral com filtros	Interface completa
/dashboard/lotes	Controle de validade de lotes	Interface completa

WMS

Rota	Descrição	Dados
/dashboard/enderecos	Endereçamento físico: corredor, rua, prateleira, nível e posição	Interface completa
/dashboard/inventario	Contagens completa, parcial e cíclica	Interface completa
/dashboard/scanner	Leitura de código de barras para rastrear produtos	Interface completa

Inteligência

Rota	Descrição	Dados
/dashboard/ia	Identificação de produtos por imagem com atributos sugeridos	Interface completa
/dashboard/ia-analitica	Previsão de demanda, matriz ABC/XYZ e sugestões de compra	Interface completa
/dashboard/relatorios	Catálogo de relatórios gerenciais	Interface completa

Administração

Rota	Descrição	Dados
/dashboard/usuarios	Gestão de usuários e perfis	API real
/dashboard/auditoria	Trilha de operações críticas	Interface completa
/dashboard/configuracoes	Empresa, preferências, aparência, notificações, segurança e integrações	Interface completa

09 Telas em destaque

● Produtos

Listagem com busca, filtro por categoria, alternância entre tabela e cartões, paginação e indicadores de situação de estoque (Em Estoque, Baixo, Crítico, Sem Estoque). Os dados vêm de `GET /products` e `GET /categories` e são convertidos para o modelo da interface por uma função de mapeamento. Exclusão usa `DELETE /products/:id` e o cadastro, `POST /products`, com seleção de categoria, marca e fornecedor carregadas da API.

Produtos

Buscar por nome, SKU ou código...

Categoria ▾

+ Novo produto

Produto	SKU	Categoria	Estoque	Situação	Ações
Smart TV 55"	SAM-TV-55	Eletrônicos	42	Em Estoque	  
Bateria 18650	BAT-1865	Energia	8	Estoque Baixo	  
Luva CA M	LUV-CA-M	EPI	0	Sem Estoque	  

Figura 4 — Esquema da listagem de produtos

● Dashboard

Reúne indicadores de produtos, valor de estoque, alertas e ocupação do armazém, gráficos de área (evolução do estoque), barras (curva ABC) e pizza (distribuição), além dos produtos mais movimentados e das movimentações recentes. Os gráficos são construídos com Recharts.

● IA Analítica



Previsão de demanda

Gráfico de linhas com histórico real e projeção para os próximos meses.



Matriz ABC × XYZ

Classifica produtos por valor (ABC) e regularidade de consumo (XYZ), com giro.



Insights e sugestões

Riscos de ruptura, sazonalidade, crescimento e lista priorizada de compras por urgência.

● Scanner

Tela voltada à operação em campo, com quatro modos — Entrada, Saída, Consulta e Transferência. O código lido (código de barras, SKU ou código interno) localiza o produto e exibe seus dados, com histórico das leituras recentes. Corresponde ao endpoint `GET /products/scan/:code` do backend.

10 Autenticação e sessão

O `AuthProvider` envolve toda a aplicação, mantém o usuário logado e expõe `login` e `logout` pelo hook `useAuth`. O token JWT é guardado no `localStorage` (chave `wms_token`) e enviado automaticamente em cada requisição.

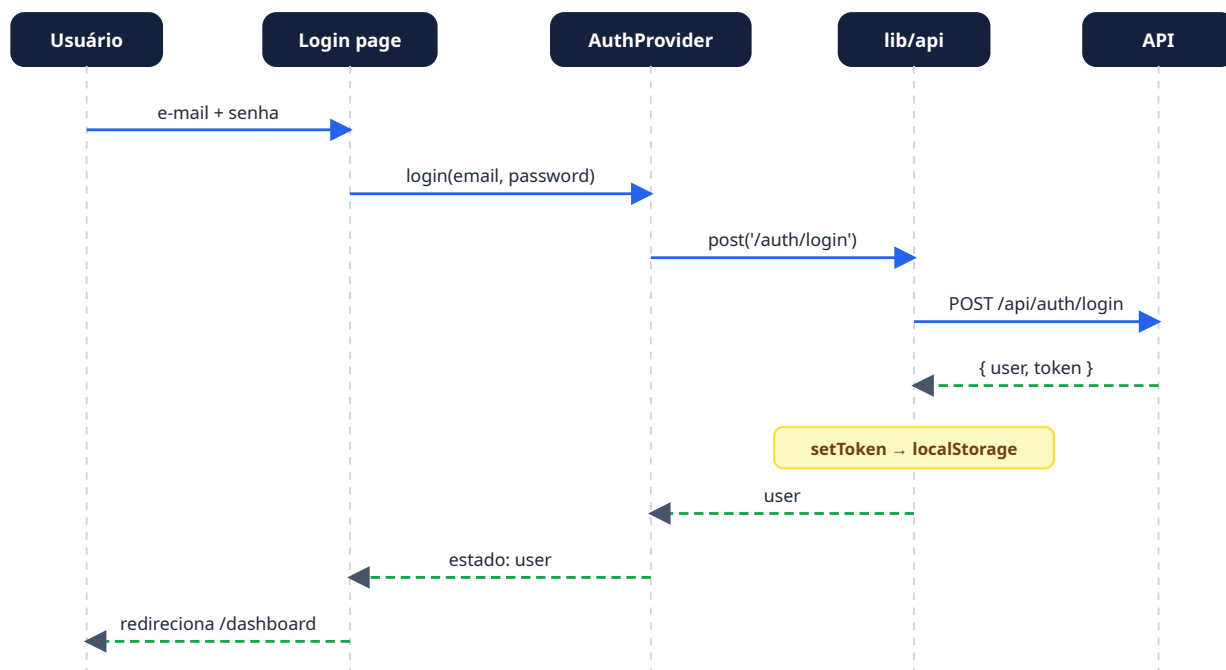


Figura 5 — Fluxo de login

■ Proteção de rotas

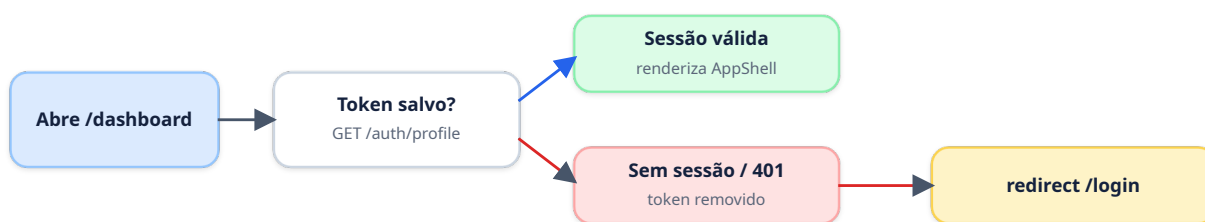


Figura 6 — Guarda de sessão do layout autenticado

O layout `(dashboard)/layout.tsx` aguarda a verificação da sessão e, se não houver usuário, redireciona para `/login`. Qualquer resposta **401** da API limpa o token e leva o usuário ao login. A sidebar exibe o perfil logado e permite sair.

11 Integração com a API

O módulo `lib/api.ts` centraliza toda a comunicação com o backend: monta a URL base, injeta o cabeçalho `Authorization`, serializa o corpo, constrói a query string e converte erros em `ApiError`.

```
const API_URL = process.env.NEXT_PUBLIC_API_URL ?? 'http://localhost:3001/api'

api.get<T>(path, params?)    // GET com query string
api.post<T>(path, body?)    // POST JSON
api.patch<T>(path, body?)   // PATCH JSON
api.put<T>(path, body?)     // PUT JSON
api.delete<T>(path)
```

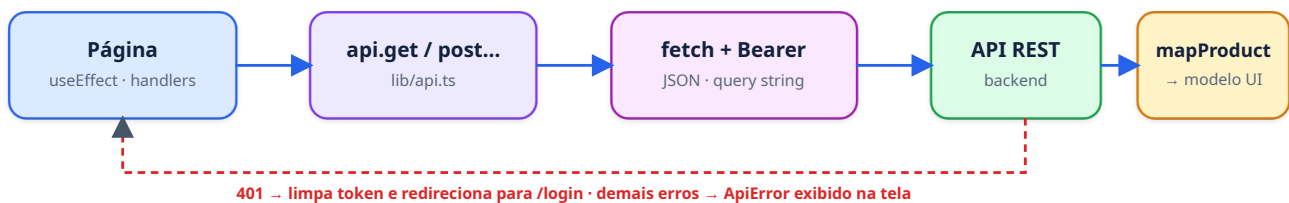


Figura 7 — Caminho de uma requisição

● Endpoints consumidos pelas telas

Tela	Operações
Login	POST /auth/login · GET /auth/profile
Produtos	GET /products · GET /categories · DELETE /products/:id
Novo produto	GET /categories · /brands · /suppliers · POST /products
Categorias	GET · POST · PATCH · DELETE /categories
Marcas	GET · POST · PATCH · DELETE /brands
Fornecedores	GET /suppliers
Clientes	GET /customers
Usuários	GET /users · POST /users

● Tipos de domínio

As tipagens em `types/` descrevem produtos, movimentações, usuários, endereços e estruturas comuns (paginação, ordenação, endereço). Respostas da API são tipadas na borda e convertidas para esses modelos, de modo que os componentes de interface independem do formato do backend. Listagens

paginadas chegam como `{ data, total, page, limit, totalPages }`; categorias e marcas chegam como arrays simples.

12 Execução e convenções

● Executando

```
cd frontend
npm install
npm run dev      # http://localhost:3000
npm run build    # build de produção
npm run start    # serve o build
npm run lint     # ESLint
```

Variável	Descrição	Padrão
NEXT_PUBLIC_API_URL	URL base da API REST	http://localhost:3001/api

ACESSO DE DEMONSTRAÇÃO

Com o backend em execução e o seed aplicado, a tela de login já vem preenchida com o usuário administrador de demonstração (`admin@wms.com`).

● Convenções de código

Tema	Padrão adotado
Rotas	App Router; <code>(auth)</code> para telas públicas e <code>(dashboard)</code> para autenticadas
Componentes	Genéricos em <code>components/ui</code> ; estrutura de página em <code>components/layout</code>
Estilos	Tailwind sobre CSS variables; sem cores fixas nos componentes
Tipos	Centralizados em <code>types/</code> , compartilhados entre telas e API
Rótulos	Dicionários em <code>constants/status.ts</code> (estoque, produto, movimentação, perfil)
Classes	<code>cn()</code> para composição condicional
Estados de tela	Carregamento (skeleton/spinner), erro e estado vazio em cada listagem
Ícones	lucide-react com traço padrão de 1.75

● Perfis e permissões na interface

As permissões são aplicadas pela API conforme o perfil do token (Visualizador, Operador, Supervisor, Administrador). Operações negadas retornam 403 e a mensagem é exibida ao usuário na própria tela. A hierarquia completa está descrita na documentação do backend.